

Deep Learning Avancé & Applications

Mini-Projet 1 : Analyse de Sentiment
avec ZenML & MLflow

Master Bac+5 – Data Analytics & Big Data

Mouaad AGOURRAM

Présentation du Projet

Le Dataset : IMDb

Orchestration avec ZenML

Tracking avec MLflow

Implémentation du Pipeline

Tests Unitaires

Livrables & Évaluation

Présentation du Projet

Ce que vous allez construire

- Un pipeline ML de bout en bout
- Analyse de sentiment sur le dataset IMDb
- Modèle : DistilBERT fine-tuné
- Orchestration et tracking automatisés

Compétences mobilisées

- Transformers (Séance 5)
- Orchestration ZenML
- Tracking MLflow
- Tests unitaires avec pytest

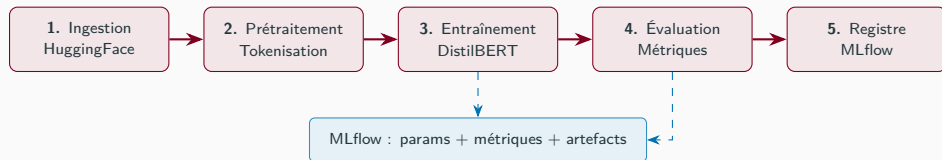
Livrable attendu

- Pipeline orchestré avec ZenML
- Expériences tracées dans MLflow
- Tests unitaires (pytest, coverage > 70%)
- Accuracy $\geq 88\%$ sur le test set IMDb

Durée estimée

4 h en séance + rendu à J+7

Pipeline ZenML — chaque boîte est un @step reproductible et versionné



DistilBERT

- Sanh et al., HuggingFace 2019
- 66 M paramètres, 40% plus léger que BERT
- Pré-entraîné Wikipedia + BooksCorpus
- Fine-tune pour classification binaire

ZenML

- Framework d'orchestration ML open-source
- @step : unité reproductible
- @pipeline : graphe de dépendances
- Artifacts versionnés automatiquement
- Compatible Kubeflow, Airflow, Vertex AI

MLflow

- Tracking des expériences
- Log params, métriques, artefacts
- Model Registry avec staging
- UI web intégrée (mlflow ui)

Le Dataset : IMDb

Maas et al., Stanford AI Lab (ACL 2011)

- 50 000 critiques de films en anglais
- 25 000 train / 25 000 test
- Classification binaire : positif / négatif
- Distribution parfaitement équilibrée (50/50)

Benchmark public

Modèle	Accuracy
BiLSTM (S4)	≈ 87%
DistilBERT fine-tuné	91,3%
BERT-base fine-tuné	93,5%

Distribution des longueurs

Longueur (tokens)	Fréq.	Avec
< 128	32%	
128 – 256	28%	
256 – 512	25%	
> 512	15%	

max_length=128 on couvre 32% sans troncature — compromis vitesse / précision.

Positif (label = 1)

"One of the best films I've seen in years. The performances are outstanding, the script is tight, and the direction flawless. A must-see."

Négatif (label = 0)

"Terrible waste of time. The plot is incoherent, the acting wooden, and the ending makes absolutely no sense. Avoid at all costs."

Défi principal : les avis longs contiennent des sentiments mixtes

"The acting was brilliant but the plot was deeply disappointing" — position et poids des mots important.

Chargement avec HuggingFace Datasets

```
1 from datasets import load_dataset
2
3 raw      = load_dataset("imdb")    # telechargement automatique
4 train_df = raw["train"].to_pandas()
5 test_df  = raw["test"].to_pandas()
6
7 # Colonnes : text (str) et label (0 ou 1)
8 print(train_df["label"].value_counts())
9 # 1      12500
10 # 0      12500
11
12 # Pour le mini-projet : sous-echantillon
13 train_df = train_df.sample(n=2000, random_state=42).reset_index(drop=True)
14 test_df  = test_df.sample(n=500,  random_state=42).reset_index(drop=True)
```

Disponibilité

Accessible via `pip install datasets` — aucun compte ni token requis. Mise en cache automatique après le premier téléchargement.

Orchestration avec ZenML

Pourquoi orchestrer ses pipelines ML ?

Sans orchestration

- Scripts isolés, non reproductibles
- Données et modèles non versionnés
- Résultats impossibles à comparer entre runs
- Onboarding difficile pour l'équipe
- Débogage laborieux en production

Avec ZenML

- Chaque étape est isolée et versionnée
- Artifacts persistés automatiquement
- Cache par défaut : replay instantané
- Interface unique quel que soit l'orchestrateur
- Logs et métadonnées pour chaque run

ZenML = infrastructure + reproductibilité + collaboration

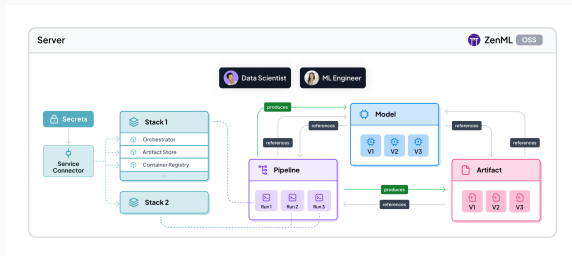
ZenML : Concepts clés

@step Unité atomique. Prend des artifacts en entrée, en produit en sortie. Versionnement automatique.

@pipeline Graphe de dépendances entre steps. ZenML infère l'ordre d'exécution.

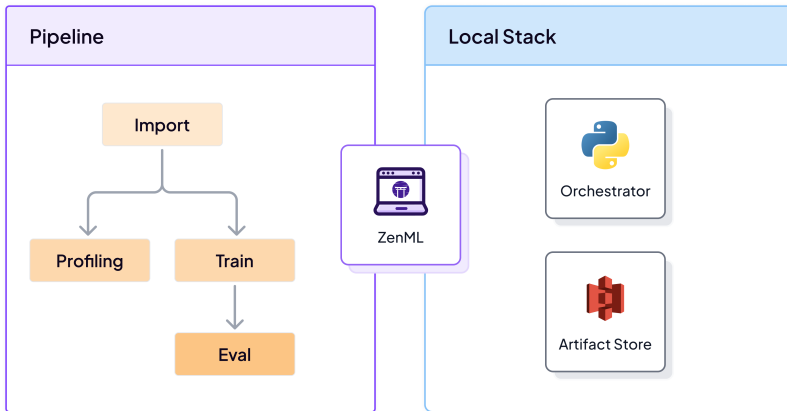
Artifact Donnée persistée entre deux steps (DataFrame, chemin modèle, dict...)

Stack Orchestrateur + artifact store + experiment tracker.



Source : docs.zenml.io — Stack, Pipeline, Model et Artifact

ZenML : Pipeline & Stack en pratique



Un pipeline ZenML s'exécute sur un **Stack** local : orchestrateur Python + artifact store.

Définir un Step

```
1 from zenml import step
2 from typing import Tuple, Annotated
3 import pandas as pd
4
5 @step
6 def load_imdb(
7     max_train_samples: int = 2000,
8     max_test_samples: int = 500,
9 ) -> Tuple[
10     Annotated[pd.DataFrame, "train_df"],
11     Annotated[pd.DataFrame, "test_df"],
12 ]:
13     from datasets import load_dataset
14     raw = load_dataset("imdb")
15     train_df = (raw["train"].to_pandas()
16                 .sample(n=max_train_samples, random_state=42)
17                 .reset_index(drop=True))
18     test_df = (raw["test"].to_pandas()
19                .sample(n=max_test_samples, random_state=42)
20                .reset_index(drop=True))
```

Définir un Pipeline

```
1 from zenml import pipeline
2 from steps.ingest      import load_imdb
3 from steps.preprocess import tokenize_data
4 from steps.train       import train_model
5 from steps.evaluate    import evaluate_model
6 from steps.register    import register_model
7
8 @pipeline(name="sentiment_analysis_pipeline")
9 def sentiment_pipeline(
10     max_train_samples: int = 2000,
11     lr:                float = 2e-5,
12     epochs:            int = 3,
13     batch_size:        int = 16,
14 ):
15     train_df, test_df = load_imdb(max_train_samples=max_train_samples)
16     train_tok, test_tok = tokenize_data(train_df, test_df)
17     model_path          = train_model(train_tok, lr=lr, epochs=epochs,
18                                     batch_size=batch_size)
19     metrics              = evaluate_model(model_path, test_tok)
20     register_model(model_path, metrics)
```


Tracking avec MLflow

Pourquoi tracker ses expériences ?

Problème classique sans tracking :

- *“Quel lr j'avais utilisé pour ce modèle qui marchait si bien ?”*
- Impossible de comparer objectivement deux runs
- Pas de traçabilité pour l'audit ou la mise en production

MLflow résout les 4 étapes du cycle de vie ML :

Tracking

Log des params, métriques et artefacts par run

Projects

Packaging reproductible du code ML

Models

Packaging et déploiement du modèle

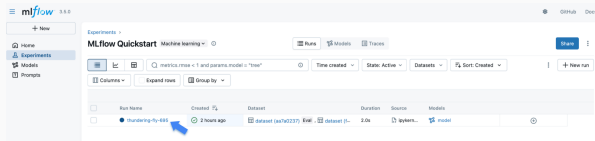
Registry

Versioning, staging et promotion

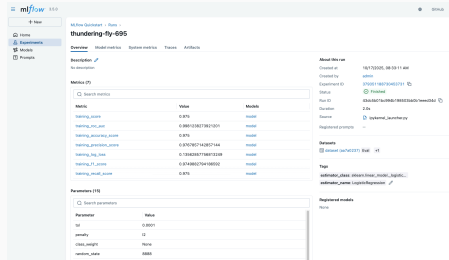
MLflow Tracking API

```
1 import mlflow
2
3 mlflow.set_tracking_uri("./mlruns")
4 mlflow.set_experiment("sentiment-analysis-imdb")
5
6 with mlflow.start_run(run_name="distilbert-lr2e-5") as run:
7     # Hyperparametres
8     mlflow.log_params({
9         "model": "distilbert-base-uncased",
10        "lr": 2e-5,
11        "epochs": 3,
12        "batch_size": 16,
13        "max_length": 128,
14    })
15
16    # Metrique par epoque
17    for epoch, loss in enumerate(train_losses):
18        mlflow.log_metric("train_loss", loss, step=epoch)
19
20    # Metriques finales
```

MLflow Tracking UI



Liste des runs d'une expérience — filtrage, tri, comparaison.



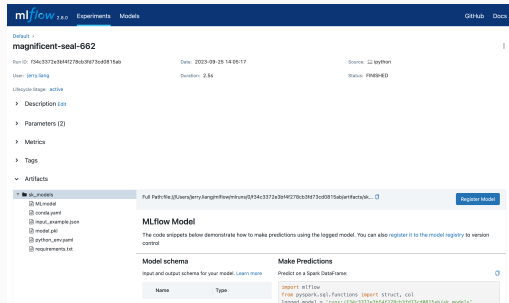
Détail d'un run : métriques, paramètres, artefacts loggés.

Source : mlflow.org/docs/latest/ml/tracking/quickstart/ (MLflow v3.5.0)

Workflow de promotion

1. Entraînement → modèle loggé dans MLflow
2. Si accuracy $\geq$ seuil : enregistrement dans Registry
3. Stages : None → Staging → Production
4. Chaque version trace ses métriques et son run_id

```
1 result = mlflow.register_model(  
2     model_uri=f"runs:{run_id}/model",  
3     name="sentiment-distilbert-imdb",  
4 )  
5 # result.version auto-incremente
```



Artefacts d'un run avec le bouton Register Model. Source : mlflow.org

Implémentation du Pipeline

Step 1 — Ingestion

```
1 @step
2 def load_imdb(
3     max_train_samples: int = 2000,
4     max_test_samples: int = 500,
5 ) -> Tuple[Annotated[pd.DataFrame, "train_df"],
6             Annotated[pd.DataFrame, "test_df"]]:
7     from datasets import load_dataset
8     raw = load_dataset("imdb")
9     train_df = (raw["train"].to_pandas()
10                 .sample(n=min(max_train_samples, 25000), random_state=42)
11                 .reset_index(drop=True))
12     test_df = (raw["test"].to_pandas()
13                .sample(n=min(max_test_samples, 25000), random_state=42)
14                .reset_index(drop=True))
15     return train_df, test_df
```

Artifact produit

Deux DataFrames (text, label) persistés dans l'artifact store — réutilisables sans re-téléchargement grâce au cache ZenML.

Step 2 — Prétraitement

```
1 from transformers import DistilBertTokenizer
2
3 @step
4 def tokenize_data(
5     train_df: pd.DataFrame, test_df: pd.DataFrame,
6     max_length: int = 128,
7 ) -> Tuple[Annotated[pd.DataFrame, "train_tokens"],
8           Annotated[pd.DataFrame, "test_tokens"]]:
9     tokenizer = DistilBertTokenizer.from_pretrained(
10         "distilbert-base-uncased"
11     )
12     for df in (train_df, test_df):
13         enc = tokenizer(
14             df["text"].tolist(),
15             padding="max_length",
16             truncation=True,
17             max_length=max_length,
18             return_tensors=None,
19         )
20         df["input_ids"] = enc["input_ids"]
```


Step 3 — Entraînement DistilBERT

```
1 @step
2 def train_model(
3     train_tokens: pd.DataFrame,
4     lr: float = 2e-5, epochs: int = 3, batch_size: int = 16,
5 ) -> str:
6     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
7     model = DistilBertForSequenceClassification.from_pretrained(
8         "distilbert-base-uncased", num_labels=2).to(device)
9     loader = DataLoader(_make_dataset(train_tokens),
10                        batch_size=batch_size, shuffle=True)
11     optimizer = AdamW(model.parameters(), lr=lr)
12     with mlflow.start_run(run_name="distilbert-train", nested=True):
13         mlflow.log_params({"lr": lr, "epochs": epochs, "bs": batch_size})
14         for epoch in range(epochs):
15             loss = _train_one_epoch(model, loader, optimizer, device)
16             mlflow.log_metric("train_loss", loss, step=epoch)
17             mlflow.log_artifacts(SAVE_DIR, artifact_path="model")
18     model.save_pretrained(SAVE_DIR)
19     return SAVE_DIR
```

Step 4 — Évaluation

```
1 @step
2 def evaluate_model(
3     model_path: str,
4     test_tokens: pd.DataFrame,
5 ) -> Dict[str, float]:
6     model =
7         DistilBertForSequenceClassification\
8             .from_pretrained(model_path)
9     preds, labels = _predict(model,
10                             test_tokens)
11     metrics = _compute_metrics(labels, preds)
12     with mlflow.start_run(nested=True):
13         mlflow.log_metrics(metrics)
14     return metrics
```

Métriques attendues

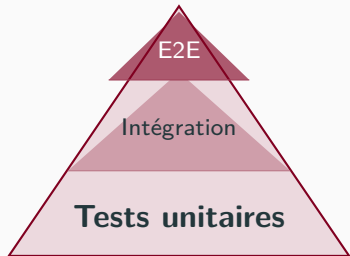
Métrique	Cible	Résultats en 3
Accuracy	$\geq 88\%$	
F1 Score	$\geq 87\%$	
Precision	$\geq 87\%$	
Recall	$\geq 87\%$	

époques, lr = 2e-5, GPU T4 (~15 min).

Step 5 — Enregistrement du Modèle

```
1 MIN_ACCURACY = 0.85
2 MODEL_NAME   = "sentiment-distilbert-imdb"
3
4 @step
5 def register_model(
6     model_path: str, metrics: Dict[str, float],
7 ) -> None:
8     if metrics.get("accuracy", 0.0) < MIN_ACCURACY:
9         logger.warning(
10             f"Modele rejete : accuracy={metrics['accuracy']:.3f} "
11             f"< seuil={MIN_ACCURACY}"
12         )
13     return
14
15 with mlflow.start_run(run_name="register") as run:
16     mlflow.log_artifacts(model_path, artifact_path="model")
17     uri      = f"runs:{run.info.run_id}/model"
18     result = mlflow.register_model(uri, name=MODEL_NAME)
19     logger.info(f"Enregistre : {MODEL_NAME} v{result.version}")
```

Tests Unitaires



Ce que nous testons

`test_ingest` Colonnes, tailles, équilibre des classes, reproductibilité

`test_preprocess` Longueur séquences, troncature, clés tokenizer

`test_evaluate` Plage $[0, 1]$, clés présentes, cas limite (parfait / inverse)

Lancer les tests

```
1 pytest pipeline/tests/ -v \  
2     --cov=steps \  
3     --cov-report=html
```

Exemple de test unitaire

```
1 # tests/test_evaluate.py
2 import pytest
3 import numpy as np
4 from steps.evaluate import _compute_metrics
5
6 def test_perfect_predictor():
7     """Un predicteur parfait doit avoir toutes les metriques a 1.0."""
8     labels = [0, 0, 1, 1, 0, 1]
9     preds = [0, 0, 1, 1, 0, 1]
10    m = _compute_metrics(labels, preds)
11    assert m["accuracy"] == pytest.approx(1.0)
12    assert m["f1"] == pytest.approx(1.0)
13    assert m["precision"] == pytest.approx(1.0)
14    assert m["recall"] == pytest.approx(1.0)
15
16 def test_metrics_in_valid_range():
17     """Toutes les metriques doivent etre dans [0, 1]."""
18     rng = np.random.default_rng(42)
19     labels = rng.integers(0, 2, size=100).tolist()
20     preds = rng.integers(0, 2, size=100).tolist()
```

Livrables & Évaluation

Ce qu'on doit retenir

Checklist livrable

- Pipeline ZenML fonctionnel (python run.py)
- MLflow UI avec ≥ 3 runs comparés (différents lr)
- Modèle enregistré dans MLflow Registry
- Tests : pytest passe avec coverage $> 70\%$
- Accuracy $\geq 88\%$ sur le test set
- Code sur GitHub (branche main)

Points clés

- **ZenML** : un @step = unité testable, cacheable, versionnée
- **MLflow** : tout hyperparamètre loggé — pas de “magic number” silencieux
- **Gate de qualité** : n'enregistrer que les modèles qui passent le seuil
- **Tests** : tester la logique pure, pas le modèle entier
- **Cache ZenML** : ne pas re-tokeniser inutilement